



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

Python 语言程序设计

基于 BERT 的新闻分类模型构建与训练

姓名 张耕嘉

年级：2023 级

专业：计算机科学与技术

2024 年 12 月 12 日

摘要

本实验旨在通过 BERT 模型对中文文本进行分类任务。实验首先通过对训练集中的文本内容和关键词进行数据预处理，包括分词、去除停用词和非中文字符，来生成模型所需的输入数据。然后，利用预训练的 bert-base-chinese 模型，设计并训练了一个文本分类模型，通过交叉熵损失函数和 AdamW 优化器进行优化。为了防止过拟合，实验中引入了早停策略，在验证集上监控模型的损失，并在验证集损失未改善时停止训练。

在训练过程中，模型的表现通过每个 epoch 的训练损失和准确度，以及验证集的损失和准确度进行评估。最终，模型在验证集上表现良好，取得了较高的准确率，并成功地对测试集进行预测。预测结果被输出并保存至 CSV 文件中。

实验结果表明，BERT 模型对于中文文本分类任务具有优越的性能，通过早停策略避免了过拟合，能够有效地处理中文文本数据。未来的研究可以尝试不同的预训练模型或数据增强方法，以进一步提升模型的分类能力和鲁棒性。

关键字：BERT 模型、分词、停用词、早停策略、文本分类、深度学习

目录

一、 概述	1
(一) 实验目的	1
(二) 实验环境	1
二、 数据处理	1
(一) 数据集	1
(二) 数据预处理	2
三、 模型构建与训练	4
(一) 模型架构	4
1. 模型原理	4
2. 代码实现	5
(二) 训练过程	6
四、 模型优化	8
(一) 学习率优化器	8
(二) 早停策略	8
(三) 训练与验证	8
五、 模型评估	9
(一) 评估指标原理及计算公式	9
(二) 实验结果	10
(三) 分析	10
(四) ROC 曲线	10
(五) 总结	11
六、 总结	11
附录	12

一、 概述

(一) 实验目的

本实验旨在通过 BERT 模型对中文文本数据进行分类任务。通过对训练集的文本内容和关键词进行处理，训练一个用于文本分类的深度学习模型，并在测试集上进行预测。具体目标包括：

- 数据预处理：分词、去停用词、去除非中文字符。
- 构建基于 BERT 的文本分类模型。
- 实现模型训练和评估，使用早停策略来防止过拟合。
- 对测试数据进行预测并输出结果。

(二) 实验环境

- 操作系统：Windows
- Python 版本：3.7
- Torch 版本：2.5.1
- transformers 版本：4.46.3
- jieba 版本：0.42.1
- CUDA：用于加速训练

二、 数据处理

(一) 数据集

本实验使用了以下数据集：

- 训练集 (`train.txt`)：包含文本内容、关键词和标签。文本内容通过分词处理并去除停用词后输入模型。
- 测试集 (`test1.txt`)：包含文本内容和关键词，用于进行预测，输出预测标签。
- 停用词文件： `cn_stopwords.txt`，用于去除常见的无意义词汇。

数据加载过程中，每条记录包括以下字段：

- ID：文本的唯一标识符。
- 文本内容：待分类的文本。
- 关键词：关键词，用于补充文本信息。
- 标签：文本的真实类别（训练集存在，测试集不存在）。

(二) 数据预处理

数据预处理的主要步骤如下：

1. 加载数据：使用 `load_data` 函数加载训练集和测试集。
2. 标签编码：将训练集中的标签转换为数值形式，方便模型训练。
3. 文本清洗：通过 `jieba` 进行分词，并去除停用词和非中文字符。
4. 文本拼接：将标题的分词结果和关键词拼接在一起，增强输入信息。
5. BERT 分词：使用 BERT 的分词器进行进一步的处理，生成 BERT 所需的输入格式 (`input_ids` 和 `attention_mask`)。

代码如下：

数据预处理

```
1 # 加载停用词
2 def load_stopwords(file_path):
3     stopwords = set()
4     with open(file_path, 'r', encoding='utf-8') as f:
5         for line in f:
6             stopwords.add(line.strip())
7     return stopwords
8
9 # 数据加载函数
10 def load_data(file_path, has_label=True):
11     data = []
12     with open(file_path, 'r', encoding='utf-8') as f:
13         for line in f:
14             try:
15                 parts = line.strip().split('_separator_')
16                 if len(parts) >= 3:
17                     id = parts[0]
18                     text = parts[1].strip() if parts[1] != 'nan' else ''
19                     key = parts[2].strip() if parts[2] != 'nan' else ''
20                     label = parts[3].strip() if has_label and len(parts) > 3
21                         and parts[3] not in ['nan', ''] else None
22                     data.append((id, text, key, label))
23             except Exception as e:
24                 print(f"Error processing line: {line} - {e}")
25     columns = ['ID', '文本内容', '关键词', '标签']
26     return pd.DataFrame(data, columns=columns)
27
28 # 正则表达式，用于保留中文字符
29 def remove_non_chinese(text):
30     return re.sub(r'[^\u4e00-\u9fa5]', '', text)
31
32
```

```
33 # 自定义 PyTorch 数据集
34 class TextDataset(Dataset):
35     def __init__(self, data, tokenizer, stopwords, max_len=128, has_label=
        True):
36         self.data = data
37         self.tokenizer = tokenizer
38         self.stopwords = stopwords
39         self.max_len = max_len
40         self.has_label = has_label
41
42     def __len__(self):
43         return len(self.data)
44
45     def __getitem__(self, index):
46         row = self.data.iloc[index]
47         # 对标题进行分词
48         title = (row['文本内容'] or '')
49         keywords = (row['关键词'] or '')
50
51         # 使用 jieba 对标题进行分词
52         title_words = jieba.cut(title)
53
54         # 将关键词按逗号分割, 不进行分词处理
55         keyword_words = keywords.split(',')
56
57         # 拼接标题分词和关键词
58         text = " ".join(title_words) + " " + " ".join(keyword_words)
59
60         # 去除停用词
61         filtered_words = [word for word in jieba.cut(text) if word not in
            self.stopwords and word.strip() != '']
62
63         # 去除非汉字字符
64         filtered_text = remove_non_chinese(" ".join(filtered_words))
65
66         # 使用 BERT 分词器处理文本
67         encoding = self.tokenizer(
68             filtered_text, # 处理好的数据
69             truncation=True,
70             padding='max_length',
71             max_length=self.max_len, # 因为标题都不长, 认为128足够, 不足128
                的部分用[PAD]填充
72             return_tensors='pt' # 返回pytorch张量, 方便处理
73         )
74         input_ids = encoding['input_ids'].squeeze()
75         attention_mask = encoding['attention_mask'].squeeze()
76         if self.has_label:
77             label = row['标签编码']
```

```
78         return input_ids, attention_mask, label
79     return input_ids, attention_mask
80
81
82 # 创建数据集和数据加载器
83 train_dataset = TextDataset(train_data, tokenizer, stopwords, has_label=True)
84 test_dataset = TextDataset(test_data, tokenizer, stopwords, has_label=False)
```

三、 模型构建与训练

(一) 模型架构

本实验采用基于 BERT 的模型作为文本分类任务的核心架构。BERT 是一种双向深层语义理解模型，基于 Transformer 编码器设计，能够在多种自然语言处理任务中取得优异的表现。本实验在预训练的 bert-base-chinese 模型基础上，通过微调实现对中文文本分类任务的适配。

1. 模型原理

BERT 模型的原理如下：

1. 模型输入 BERT 模型的输入由以下三部分嵌入信息组成：

- (a) Token Embeddings：将分词后的 Token 映射为高维向量，通过查找预训练嵌入矩阵获取固定大小的表示。
- (b) Segment Embeddings：用于区分句子对中的句子 A 和句子 B，为每个 Token 添加对应的嵌入向量。
- (c) Position Embeddings：为每个 Token 提供位置信息，通过训练学习到独特的嵌入向量。

此外，BERT 引入特殊标志：

CLS ：表示句子开始，提供句子全局表示（通常用于分类任务）。

SEP ：分隔句子对或标志序列结束。

PAD ：用于对齐不同长度的序列，便于批量处理。

2. Transformer 架构

BERT 模型的核心是 Transformer 编码器（Encoder）。Transformer 的编码器由以下部分组成：

多头自注意力机制（Multi-head Self-Attention）：自注意力机制通过计算序列中每个词对其他词的注意力分数，捕捉全局上下文信息。多头机制允许模型关注不同的语义关系，增强了语义表示能力。

前馈神经网络（Feed-Forward Neural Network, FFN）：对每个词的表示进行非线性变换，进一步提取特征。

残差连接和层归一化（Residual Connection and Layer Normalization）：提高模型的训练稳定性，加速收敛。

3. 双向语义建模

BERT 通过双向 Transformer 编码器，同时从左到右和从右到左建模上下文信息。这种双向建模能力使 BERT 能够更好地理解文本的语义，尤其是在长文本或复杂句子中。

4. 预训练任务 BERT 通过以下两种任务在大规模语料上进行预训练：

掩码语言模型（Masked Language Model, MLM）：随机屏蔽部分输入词（例如，[MASK]），让模型根据上下文预测被屏蔽的词。

下一句预测（Next Sentence Prediction, NSP）：判断两段输入文本是否是连续的，以学习句子间的关系。

5. 迁移学习 BERT 经过预训练后，可以通过添加任务特定的头部（如分类层）进行微调，快速适应不同的下游任务，例如分类、问答、翻译等。

6. 全连接分类头部在本实验中，使用 BertForSequenceClassification 作为基础模型，该模型在预训练的 bert-base-chinese 模型基础上添加了一个全连接分类头部。

2. 代码实现

(a) 输入层

输入数据经过预处理后转化为 BERT 模型可接受的格式：

- input_ids：每个词对应的词汇表索引。
- attention_mask：标记有效输入位置，填充部分为 0。
- token_type_ids（本实验未用到）：区分句子对任务中的不同句子。

在代码中，输入层的实现如下：

输入层实现

```
1  # 使用BERT分词器对文本进行分词并生成输入格式
2  encoding = tokenizer(
3      filtered_text, # 预处理后的文本
4      truncation=True,
5      padding='max_length',
6      max_length=self.max_len,
7      return_tensors='pt' # 返回PyTorch张量
8  )
9  input_ids = encoding['input_ids'].squeeze() # 文本索引
10 attention_mask = encoding['attention_mask'].squeeze() # 注意力掩码
```

(b) BERT 主干模型

BERT 主干由 12 层 Transformer 编码器组成，每层包含多头自注意力机制和前馈神经网络。输入数据经过 BERT 主干后，输出以下特征：

句子向量（[CLS] 标记向量）：表示整段输入文本的语义信息，通常用于分类任务。

词向量：表示输入序列中每个词的语义特征。

代码中通过加载预训练的 bert-base-chinese 模型实现：

加载预训练模型

```

1 from transformers import BertForSequenceClassification
2
3 # 加载预训练的BERT模型，指定分类类别数
4 model = BertForSequenceClassification.from_pretrained(
5     'bert-base-chinese',
6     num_labels=len(label_to_index) # 分类数
7 )
8 model.to(device) # 将模型移动到GPU或CPU

```

(c) 分类头部

分类头部是一个简单的全连接层，用于将 BERT 的输出映射到特定任务的类别空间：

输入：BERT 输出的 [CLS] 标记向量，维度为 768。

输出：分类任务的类别数，例如 3 分类任务的输出为 3 维向量。

Softmax 层：将输出转化为每个类别的概率分布。

分类头部的实现由 BertForSequenceClassification 模型自动完成，无需额外代码。

(d) 损失函数

损失函数使用交叉熵损失 (CrossEntropyLoss)，用于衡量预测值与真实标签之间的差距：

交叉熵损失

```

1 from torch.nn import CrossEntropyLoss
2
3 loss_fn = CrossEntropyLoss()
4 loss = loss_fn(logits, labels) # logits是模型输出，labels是真实标签

```

(二) 训练过程

训练过程包括以下几个步骤：

1. 训练函数：train_epoch 函数负责执行一个 epoch 的训练，计算损失和准确度，并更新模型参数。

```

1 def train_epoch(model, data_loader, optimizer, device):
2     model.train()
3     loss_fn = CrossEntropyLoss()
4     total_loss, total_correct = 0, 0
5
6     for input_ids, attention_mask, labels in tqdm(data_loader):
7         input_ids, attention_mask, labels = (
8             input_ids.to(device),
9             attention_mask.to(device),
10            labels.to(device)
11        )
12        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
13        logits = outputs.logits
14        loss = loss_fn(logits, labels)

```

```

15         total_loss += loss.item()
16         total_correct += (logits.argmax(dim=1) == labels).sum().item()
17
18         optimizer.zero_grad()
19         loss.backward()
20         optimizer.step()
21
22         # 清理缓存
23         torch.cuda.empty_cache()
24     return total_loss / len(data_loader), total_correct / len(data_loader.
        dataset)

```

2. 验证函数：eval_model 函数负责在验证集上进行评估，计算损失和准确度。

```

26 def eval_model(model, data_loader, device):
27     model.eval()
28     total_loss, total_correct = 0, 0
29     loss_fn = CrossEntropyLoss()
30
31     all_true_labels = []
32     all_predicted_labels = []
33     all_predicted_probs = []
34
35     with torch.no_grad():
36         for input_ids, attention_mask, labels in data_loader:
37             input_ids, attention_mask, labels = (
38                 input_ids.to(device),
39                 attention_mask.to(device),
40                 labels.to(device)
41             )
42             outputs = model(input_ids=input_ids, attention_mask=
                attention_mask)
43             logits = outputs.logits
44             loss = loss_fn(logits, labels)
45             total_loss += loss.item()
46             total_correct += (logits.argmax(dim=1) == labels).sum().item()
47
48             all_true_labels.extend(labels.cpu().numpy())
49             all_predicted_labels.extend(logits.argmax(dim=1).cpu().numpy())
50             all_predicted_probs.extend(torch.softmax(logits, dim=1).cpu().
                numpy())
51
52     return total_loss / len(data_loader), all_true_labels,
        all_predicted_labels, all_predicted_probs

```

3. 早停策略：为了防止过拟合，使用早停策略。当验证集损失连续若干轮未下降时，停止训练。

训练过程中，打印出每个 epoch 的训练损失、训练准确度、验证损失和验证准确度。

运行过程截图如下：

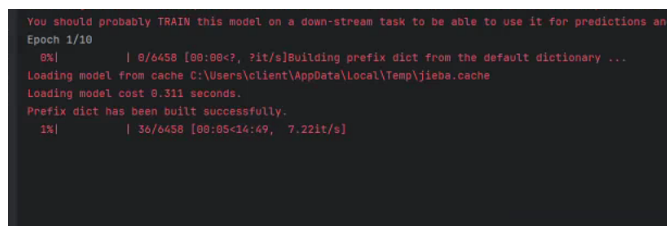


图 1: 运行过程截图

四、 模型优化

(一) 学习率优化器

采用 AdamW 优化器, 这是适用于 Transformer 模型的一种优化算法, 支持权重衰减 (weight decay), 可以有效防止过拟合:

```
1 from torch.optim import AdamW
2
3 optimizer = AdamW(model.parameters(), lr=2e-5) # 学习率为2e-5
```

(二) 早停策略

实验中引入了早停策略以防止过拟合。当验证集损失连续若干轮没有下降时, 停止训练并保存最优模型:

```
1 if val_loss < best_val_loss:
2     best_val_loss = val_loss
3     best_model_state = model.state_dict() # 保存最优模型
4     patience_counter = 0
5 else:
6     patience_counter += 1
7 if patience_counter >= patience:
8     print("Early stopping triggered.")
9     break
```

(三) 训练与验证

- 训练过程: 模型处于训练模式 (`model.train()`), 通过前向传播计算损失, 反向传播更新参数。
- 验证过程: 模型处于评估模式 (`model.eval()`), 在验证集上评估损失和准确率。

```
1 def train_epoch(model, data_loader, optimizer, device):
2     model.train()
3     for input_ids, attention_mask, labels in tqdm(data_loader):
4         input_ids, attention_mask, labels = (
```

```
5         input_ids.to(device),
6         attention_mask.to(device),
7         labels.to(device)
8     )
9     outputs = model(input_ids=input_ids, attention_mask=attention_mask)
10    logits = outputs.logits
11    loss = loss_fn(logits, labels)
12    optimizer.zero_grad()
13    loss.backward()
14    optimizer.step()
```

五、 模型评估

为了全面衡量模型的分类性能，本实验使用了以下常用评估指标：准确率（Accuracy）、精确率（Precision）、召回率（Recall）、F1 分数（F1-Score）以及 ROC 曲线下的面积（AUC）。这些指标从不同角度评估了模型在测试集上的表现。

（一） 评估指标原理及计算公式

（1） 准确率（Accuracy）

定义：表示模型正确预测的样本数量占总样本数量的比例。**公式：**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

其中：

- TP ：真阳性，模型正确预测为正的数量。
- TN ：真阴性，模型正确预测为负的数量。
- FP ：假阳性，模型错误预测为正的数量。
- FN ：假阴性，模型错误预测为负的数量。

（2） 精确率（Precision）

定义：在模型预测为正的样本中，实际为正的样本比例。**公式：**

$$Precision = \frac{TP}{TP + FP}$$

（3） 召回率（Recall）

定义：在实际为正的样本中，模型正确预测为正的样本比例。**公式：**

$$Recall = \frac{TP}{TP + FN}$$

（4） F1 分数（F1-Score）

定义：精确率和召回率的调和平均，用于综合评价模型性能。**公式：**

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

(5) ROC-AUC

定义：ROC 曲线下的面积，反映模型在不同阈值下的整体分类性能。**特点：**AUC 值越接近 1，模型性能越优。

(二) 实验结果

测试集上的评估指标为：

Accuracy = 90.11%

Precision = 90.05%

Recall = 90.11%

F1-Score = 90.05%

(三) 分析

- **准确率：**准确率达到 90.11%，表明模型能够较准确地对大多数样本进行分类。准确率的高值反映了模型对整体分类任务的良好适配性。
- **精确率：**精确率与准确率相近，说明模型的预测结果中，误报率较低。这表明模型对正样本的预测可靠性较高。
- **召回率：**召回率高，表明模型在对各类别的正样本检测方面表现优秀。与精确率的相对均衡说明了模型在分类时没有明显的类别偏向，正负样本预测平衡性较好。
- **F1 分数：**F1-Score 较高，且与精确率和召回率接近，表明模型在正样本检测上实现了良好的平衡。高 F1-Score 说明模型的误报率和漏报率均较低，综合性能优越。
- **AUC 值：**所有 15 个分类的 AUC 均接近 1，说明模型在各类别上的分类能力都非常强。这一结果反映了模型对不同类别具有一致的区分能力，没有特定类别的明显弱势。

(四) ROC 曲线

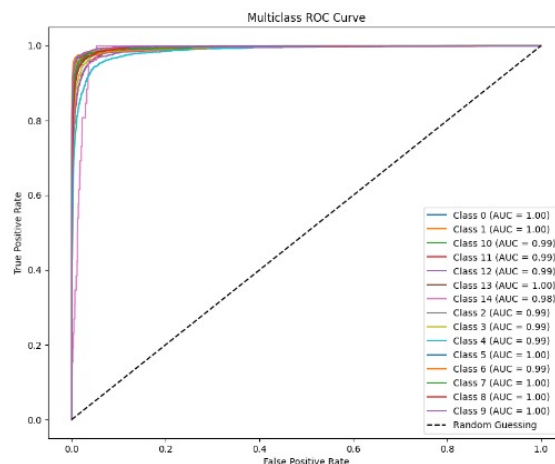


图 2: 测试集上的 ROC 曲线及 AUC 值

(五) 总结

通过以上指标的评估可以看出，本实验中的 BERT 模型在测试集上的分类性能优越，能够很好地完成新闻文本的分类任务。模型的高准确率和 AUC 值表明其具有良好的整体性能，F1 分数反映了精确率和召回率的平衡能力。未来可以尝试进一步优化召回率，以更全面地提升分类性能。

六、 总结

本实验成功实现了中文文本分类任务，并通过 BERT 模型取得了较好的结果。实验结论如下：

- 使用 BERT 模型对于文本分类任务具有良好的性能，能够有效处理中文文本。
- 早停策略有效地避免了过拟合，使得模型在验证集上有较好的表现。
- 在处理大型数据集时，使用 GPU 加速训练能够显著提高效率。

未来的改进方向包括：

- 尝试其他预训练模型，如 RoBERTa、ERNIE 等，以进一步提高分类精度。
- 可以在文本清洗阶段引入更多的数据增强方法，增强模型的鲁棒性。
- 针对更大规模的数据集，优化训练过程的效率。

附录

完整代码

BERTpytorch+GPU4.py

```

1 import jieba
2 import re
3 import pandas as pd
4 import torch
5 from torch.utils.data import Dataset, DataLoader, random_split
6 from torch.optim import AdamW
7 from torch.nn import CrossEntropyLoss
8 from transformers import BertTokenizer, BertForSequenceClassification
9 from tqdm import tqdm
10 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve, auc
11 import matplotlib.pyplot as plt
12 import numpy as np
13
14 # 加载停用词
15 def load_stopwords(file_path):
16     stopwords = set()
17     with open(file_path, 'r', encoding='utf-8') as f:
18         for line in f:
19             stopwords.add(line.strip())
20     return stopwords
21
22
23 # 数据加载函数
24 def load_data(file_path, has_label=True):
25     data = []
26     with open(file_path, 'r', encoding='utf-8') as f:
27         for line in f:
28             try:
29                 parts = line.strip().split('_separator_')
30                 if len(parts) >= 3:
31                     id = parts[0]
32                     text = parts[1].strip() if parts[1] != 'nan' else ''
33                     key = parts[2].strip() if parts[2] != 'nan' else ''
34                     label = parts[3].strip() if has_label and len(parts) > 3
35                         and parts[3] not in ['nan', ''] else None
36                     data.append((id, text, key, label))
37             except Exception as e:
38                 print(f"Error processing line: {line} - {e}")
39     columns = ['ID', '文本内容', '关键词', '标签']
40     return pd.DataFrame(data, columns=columns)
41

```

```
42 # 加载数据
43 train_data = load_data('train.txt', has_label=True)
44 test_data = load_data('test1.txt', has_label=False)
45
46 # 标签编码
47 unique_labels = sorted(set(train_data['标签'].dropna()))
48 label_to_index = {label: idx for idx, label in enumerate(unique_labels)}
49 index_to_label = {idx: label for label, idx in label_to_index.items()}
50 train_data['标签编码'] = train_data['标签'].map(label_to_index)
51
52 # 加载 BERT 分词器
53 tokenizer = BertTokenizer.from_pretrained('bert-base-chinese')
54
55 # 加载停用词
56 stopwords = load_stopwords('cn_stopwords.txt')
57
58
59 # 正则表达式, 用于保留中文字符
60 def remove_non_chinese(text):
61     return re.sub(r'[^\u4e00-\u9fa5]', '', text)
62
63
64 # 自定义 PyTorch 数据集
65 class TextDataset(Dataset):
66     def __init__(self, data, tokenizer, stopwords, max_len=128, has_label=
        True):
67         self.data = data
68         self.tokenizer = tokenizer
69         self.stopwords = stopwords
70         self.max_len = max_len
71         self.has_label = has_label
72
73     def __len__(self):
74         return len(self.data)
75
76     def __getitem__(self, index):
77         row = self.data.iloc[index]
78         # 对标题进行分词
79         title = (row['文本内容'] or '')
80         keywords = (row['关键词'] or '')
81
82         # 使用 jieba 对标题进行分词
83         title_words = jieba.cut(title)
84
85         # 将关键词按逗号分割, 不进行分词处理
86         keyword_words = keywords.split(',')
87
88         # 拼接标题分词和关键词
```



```

89         text = " ".join(title_words) + " " + " ".join(keyword_words)
90
91         # 去除停用词
92         filtered_words = [word for word in jieba.cut(text) if word not in
93                             self.stopwords and word.strip() != '']
94
95         # 去除非汉字字符
96         filtered_text = remove_non_chinese(" ".join(filtered_words))
97
98         # 使用 BERT 分词器处理文本
99         encoding = self.tokenizer(
100             filtered_text,      # 处理好的数据
101             truncation=True,
102             padding='max_length',
103             max_length=self.max_len, # 因为标题都不长, 认为128足够, 不足128
104                                     # 的部分用[PAD]填充
105             return_tensors='pt'      # 返回pytorch张量, 方便处理
106         )
107         input_ids = encoding['input_ids'].squeeze()
108         attention_mask = encoding['attention_mask'].squeeze()
109         if self.has_label:
110             label = row['标签编码']
111             return input_ids, attention_mask, label
112         return input_ids, attention_mask
113
114 # 创建数据集和数据加载器
115 train_dataset = TextDataset(train_data, tokenizer, stopwords, has_label=True)
116 test_dataset = TextDataset(test_data, tokenizer, stopwords, has_label=False)
117
118 train_size = int(0.9 * len(train_dataset))
119 val_size = len(train_dataset) - train_size
120 train_dataset, val_dataset = random_split(train_dataset, [train_size,
121                                                         val_size])
122
123 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
124 val_loader = DataLoader(val_dataset, batch_size=32)
125 test_loader = DataLoader(test_dataset, batch_size=32)
126
127 # 加载模型
128 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
129 model = BertForSequenceClassification.from_pretrained('bert-base-chinese',
130                                                       num_labels=len(label_to_index))
131 model.to(device) # 将模型移到 GPU 或 CPU 上
132
133 # 优化器
134 optimizer = AdamW(model.parameters(), lr=2e-5)

```

```

133
134 # 训练函数
135 def train_epoch(model, data_loader, optimizer, device):
136     model.train()
137     loss_fn = CrossEntropyLoss()
138     total_loss, total_correct = 0, 0
139
140     for input_ids, attention_mask, labels in tqdm(data_loader):
141         input_ids, attention_mask, labels = (
142             input_ids.to(device),
143             attention_mask.to(device),
144             labels.to(device)
145         )
146         outputs = model(input_ids=input_ids, attention_mask=attention_mask)
147         logits = outputs.logits
148         loss = loss_fn(logits, labels)
149         total_loss += loss.item()
150         total_correct += (logits.argmax(dim=1) == labels).sum().item()
151
152         optimizer.zero_grad()
153         loss.backward()
154         optimizer.step()
155
156     # 清理缓存
157     torch.cuda.empty_cache()
158     return total_loss / len(data_loader), total_correct / len(data_loader.
159         dataset)
160
161 def plot_multiclass_roc(y_true, y_probs, num_classes):
162     true_labels_one_hot = np.zeros((len(y_true), num_classes))
163     for i, label in enumerate(y_true):
164         true_labels_one_hot[i, label] = 1
165
166     plt.figure(figsize=(10, 8))
167     for i in range(num_classes):
168         fpr, tpr, _ = roc_curve(true_labels_one_hot[:, i], y_probs[:, i])
169         roc_auc = auc(fpr, tpr)
170         plt.plot(fpr, tpr, label=f'Class_{index_to_label[i]} (AUC={roc_auc
171             :.2f})')
172
173     plt.plot([0, 1], [0, 1], 'k--', label='Random_Guessing')
174     plt.xlabel('False_Positive_Rate')
175     plt.ylabel('True_Positive_Rate')
176     plt.title('Multiclass ROC Curve')
177     plt.legend(loc='lower_right')
178     plt.show()

```

```

179
180 # 验证函数（移除即时评估指标输出，返回完整数据）
181 def eval_model(model, data_loader, device):
182     model.eval()
183     total_loss, total_correct = 0, 0
184     loss_fn = CrossEntropyLoss()
185
186     all_true_labels = []
187     all_predicted_labels = []
188     all_predicted_probs = []
189
190     with torch.no_grad():
191         for input_ids, attention_mask, labels in data_loader:
192             input_ids, attention_mask, labels = (
193                 input_ids.to(device),
194                 attention_mask.to(device),
195                 labels.to(device)
196             )
197             outputs = model(input_ids=input_ids, attention_mask=
198                             attention_mask)
199             logits = outputs.logits
200             loss = loss_fn(logits, labels)
201             total_loss += loss.item()
202             total_correct += (logits.argmax(dim=1) == labels).sum().item()
203
204             all_true_labels.extend(labels.cpu().numpy())
205             all_predicted_labels.extend(logits.argmax(dim=1).cpu().numpy())
206             all_predicted_probs.extend(torch.softmax(logits, dim=1).cpu().
207                                       numpy())
208
209     return total_loss / len(data_loader), all_true_labels,
210           all_predicted_labels, all_predicted_probs
211
212 # 训练模型
213 EPOCHS = 10# 验证函数（移除即时评估指标输出，返回完整数据）
214 def eval_model(model, data_loader, device):
215     model.eval()
216     total_loss, total_correct = 0, 0
217     loss_fn = CrossEntropyLoss()
218
219     all_true_labels = []
220     all_predicted_labels = []
221     all_predicted_probs = []
222
223     with torch.no_grad():
224         for input_ids, attention_mask, labels in data_loader:
225             input_ids, attention_mask, labels = (

```

```

224         input_ids.to(device),
225         attention_mask.to(device),
226         labels.to(device)
227     )
228     outputs = model(input_ids=input_ids, attention_mask=
        attention_mask)
229     logits = outputs.logits
230     loss = loss_fn(logits, labels)
231     total_loss += loss.item()
232     total_correct += (logits.argmax(dim=1) == labels).sum().item()
233
234     all_true_labels.extend(labels.cpu().numpy())
235     all_predicted_labels.extend(logits.argmax(dim=1).cpu().numpy())
236     all_predicted_probs.extend(torch.softmax(logits, dim=1).cpu().
        numpy())
237
238     return total_loss / len(data_loader), all_true_labels,
        all_predicted_labels, all_predicted_probs
239
240 patience = 3 # 设定耐心值为3轮
241 best_val_loss = float('inf')
242 patience_counter = 0 # 记录验证集损失没有改善的轮次
243 best_model_state = None # 用于保存最优模型
244
245 final_true_labels = None
246 final_predicted_probs = None
247 final_predicted_labels = None
248
249 for epoch in range(EPOCHS):
250     print(f"Epoch_{epoch+1}/{EPOCHS}")
251     train_loss, train_acc = train_epoch(model, train_loader, optimizer,
        device)
252     val_loss, all_true_labels, all_predicted_labels, all_predicted_probs =
        eval_model(model, val_loader, device)
253
254     # 计算分类指标
255     val_acc = accuracy_score(all_true_labels, all_predicted_labels)
256     val_precision = precision_score(all_true_labels, all_predicted_labels,
        average='weighted')
257     val_recall = recall_score(all_true_labels, all_predicted_labels, average=
        'weighted')
258     val_f1 = f1_score(all_true_labels, all_predicted_labels, average='
        weighted')
259
260     print(f"Train_Loss:_{train_loss:.4f},_Train_Accuracy:_{train_acc:.4f}")
261     print(f"Val_Loss:_{val_loss:.4f},_Val_Accuracy:_{val_acc:.4f}")
262     print(f"Val_Precision:_{val_precision:.4f},_Val_Recall:_{val_recall:.4f},
        _Val_F1_Score:_{val_f1:.4f}")

```

```

263
264 # 如果验证集损失下降, 保存模型并重置耐心计数器
265 if val_loss < best_val_loss:
266     best_val_loss = val_loss
267     best_model_state = model.state_dict() # 保存最优模型状态
268     patience_counter = 0 # 重置耐心计数器
269
270     # 保存最优模型的验证集结果
271     final_true_labels = all_true_labels
272     final_predicted_probs = all_predicted_probs
273     final_predicted_labels = all_predicted_labels
274 else:
275     patience_counter += 1 # 增加耐心计数器
276
277 # 如果耐心计数器达到设定值, 停止训练
278 if patience_counter >= patience:
279     print(f"Early stopping triggered. Stopping training after {epoch+1}
280           epochs.")
281     break
282
283 # 恢复到最佳模型状态
284 if best_model_state is not None:
285     model.load_state_dict(best_model_state)
286
287 # 在训练结束后基于最优模型计算最终指标并绘制ROC曲线
288 if final_true_labels is not None and final_predicted_probs is not None:
289     final_accuracy = accuracy_score(final_true_labels, final_predicted_labels
290                                     )
291     final_precision = precision_score(final_true_labels,
292                                     final_predicted_labels, average='weighted')
293     final_recall = recall_score(final_true_labels, final_predicted_labels,
294                                average='weighted')
295     final_f1 = f1_score(final_true_labels, final_predicted_labels, average='
296                        weighted')
297
298     print("\nFinal Evaluation Metrics:")
299     print(f"Accuracy: {final_accuracy:.4f}")
300     print(f"Precision: {final_precision:.4f}")
301     print(f"Recall: {final_recall:.4f}")
302     print(f"F1 Score: {final_f1:.4f}")
303
304     # 绘制最终的ROC曲线
305     plot_multiclass_roc(final_true_labels, np.array(final_predicted_probs),
306                        len(unique_labels))
307
308 # 预测测试集

```

```
305 model.eval()
306 predicted_labels = []
307
308 with torch.no_grad():
309     for input_ids, attention_mask in tqdm(test_loader):
310         input_ids, attention_mask = input_ids.to(device), attention_mask.to(
            device)
311         outputs = model(input_ids=input_ids, attention_mask=attention_mask)
312         logits = outputs.logits
313         predicted_labels.extend(logits.argmax(dim=1).cpu().numpy())
314
315 # 将预测结果映射回标签
316 test_data['预测标签'] = [index_to_label[label] for label in predicted_labels]
317
318 # 保存预测结果
319 test_data[['ID', '预测标签']].to_csv("submission.csv", index=False)
320
321 # 输出部分预测结果
322 print(test_data[['ID', '预测标签']].head())
323
324
325 # 保存模型
326 torch.save(model.state_dict(), 'best_model_zgj.pth')
```